

AI ASSISTANT CODING 12.4

Name: R. Pranitha Reddy

H.T.NO:2303A52248

Batch:43

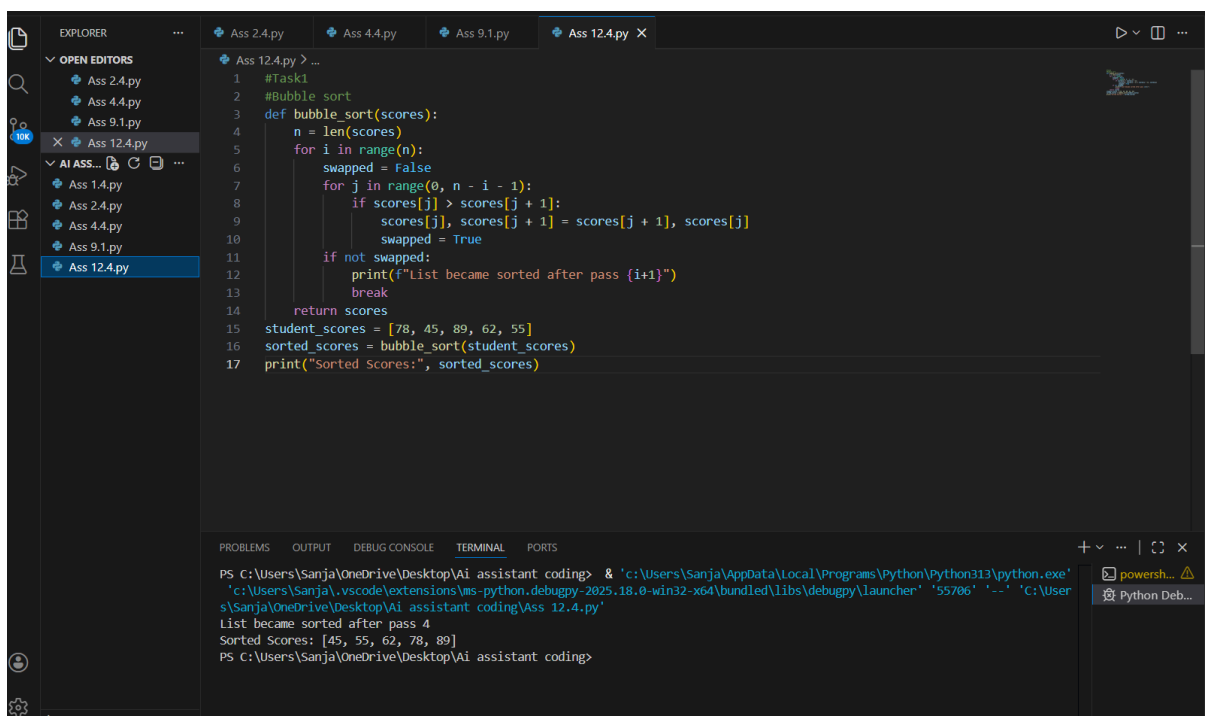
Task1:

Write a Python program to implement Bubble Sort for sorting student exam scores.

Add inline comments explaining comparisons, swaps, and passes.

Include early termination using a swapped flag.

Provide sample input/output and briefly explain best, average, and worst-case time complexity.



```
1 #Task1
2 #Bubble sort
3 def bubble_sort(scores):
4     n = len(scores)
5     for i in range(n):
6         swapped = False
7         for j in range(0, n - i - 1):
8             if scores[j] > scores[j + 1]:
9                 scores[j], scores[j + 1] = scores[j + 1], scores[j]
10                swapped = True
11        if not swapped:
12            print(f"List became sorted after pass {i+1}")
13            break
14    return scores
15 student_scores = [78, 45, 89, 62, 55]
16 sorted_scores = bubble_sort(student_scores)
17 print("Sorted Scores:", sorted_scores)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> & 'c:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe'
'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '55706' '-' 'C:\User
s\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 12.4.py'
List became sorted after pass 4
Sorted Scores: [45, 55, 62, 78, 89]
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
```

Observation:

Bubble Sort was implemented to sort student scores after internal assessments.

The algorithm shows $O(n)$ time complexity in the best case when the list is already sorted due to early termination.

In the average case, when scores are randomly arranged, the time complexity is $O(n^2)$.

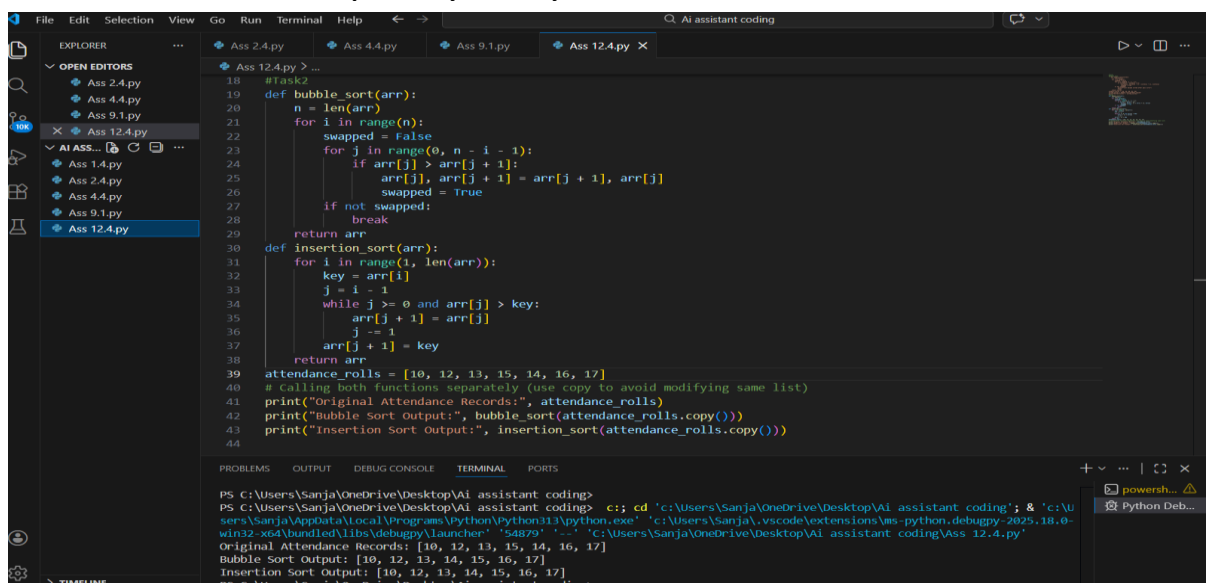
In the worst case, when scores are in reverse order, it also takes $O(n^2)$ time due to maximum comparisons and swaps.

Thus, Bubble Sort is suitable for small datasets but not efficient for large datasets.

Task2:

I have a nearly sorted list of student roll numbers (attendance records).

1. Implement Bubble Sort in Python.
2. Suggest a more suitable sorting algorithm for nearly sorted data.
3. Implement Insertion Sort in Python.
4. Explain why Insertion Sort performs better on nearly sorted datasets.
5. Compare their behavior on a nearly sorted input example.
6. Include time complexity analysis.



```
18 #Task2
19 def bubble_sort(arr):
20     n = len(arr)
21     for i in range(n):
22         swapped = False
23         for j in range(0, n - i - 1):
24             if arr[j] > arr[j + 1]:
25                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
26                 swapped = True
27         if not swapped:
28             break
29     return arr
30
31 def insertion_sort(arr):
32     for i in range(1, len(arr)):
33         key = arr[i]
34         j = i - 1
35         while j >= 0 and arr[j] > key:
36             arr[j + 1] = arr[j]
37             j -= 1
38         arr[j + 1] = key
39     return arr
40
41 attendance_rolls = [10, 12, 13, 15, 14, 16, 17]
42 # Calling both functions separately (use copy to avoid modifying same list)
43 print("Original Attendance Records:", attendance_rolls)
44 print("Bubble Sort Output:", bubble_sort(arr.copy()))
45 print("Insertion Sort Output:", insertion_sort(arr.copy()))
46
```

```
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding>
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding> c:\cd 'c:\Users\Sanja\OneDrive\Desktop\AI assistant coding' & 'c:\u
ser\Sanja\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\Sanja\vscode\extensions\ms-python-debugpy-2025.18.0-
win32-x64\bundled\libs\debugpy\launcher' '54879' '-c' 'C:\Users\Sanja\OneDrive\Desktop\AI assistant coding\Ass 12.4.py'
Original Attendance Records: [10, 12, 13, 15, 14, 16, 17]
Bubble Sort Output: [10, 12, 13, 14, 15, 16, 17]
Insertion Sort Output: [10, 12, 13, 14, 15, 16, 17]
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding>
```

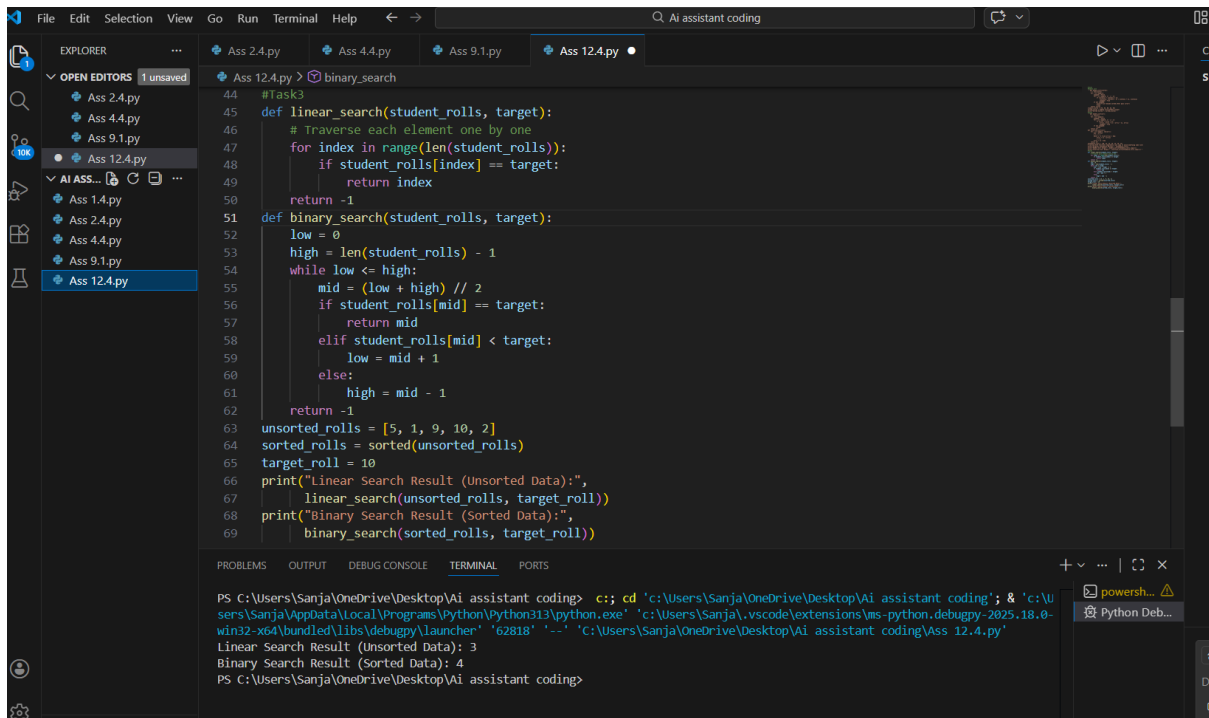
Observation:

- 1.The attendance roll numbers were nearly sorted with only a few misplaced elements.
- 2.Bubble Sort performed multiple comparisons even when only small corrections were needed.
- 3.Insertion Sort shifted only the misplaced elements to their correct positions.
- 4.For nearly sorted data, Insertion Sort behaved close to **$O(n)$** time complexity.
5. Bubble Sort still required more passes, making it comparatively slower.
- 6.Therefore, Insertion Sort is more efficient and suitable for partially sorted attendance records.

Task3:

Building a student information portal.

1. Implement Linear Search for unsorted student roll numbers.
2. Implement Binary Search for sorted roll numbers.
3. Add proper Python docstrings explaining parameters and return values.
4. Explain when Binary Search can be used.
5. Compare time complexity and performance differences.
6. Provide sample input and output.
7. Add a short observation comparing results on sorted vs unsorted data.



```
44 #Task3
45 def linear_search(student_rolls, target):
46     # Traverse each element one by one
47     for index in range(len(student_rolls)):
48         if student_rolls[index] == target:
49             return index
50     return -1
51 def binary_search(student_rolls, target):
52     low = 0
53     high = len(student_rolls) - 1
54     while low <= high:
55         mid = (low + high) // 2
56         if student_rolls[mid] == target:
57             return mid
58         elif student_rolls[mid] < target:
59             low = mid + 1
60         else:
61             high = mid - 1
62     return -1
63 unsorted_rolls = [5, 1, 9, 10, 2]
64 sorted_rolls = sorted(unsorted_rolls)
65 target_roll = 10
66 print("Linear Search Result (Unsorted Data):",
67       linear_search(unsorted_rolls, target_roll))
68 print("Binary Search Result (Sorted Data):",
69       binary_search(sorted_rolls, target_roll))
```

```
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c:: cd 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\U
ers\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-
win32-x64\bundle\libs\debugpy\launcher' '62818' '--' 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 12.4.py'
Linear Search Result (Unsorted Data): 3
Binary Search Result (Sorted Data): 4
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
```

Observation:

Linear Search checks each element one by one, so its time complexity is **$O(n)$** in average and worst cases. It works on both sorted and unsorted lists.

Binary Search divides the sorted list into halves repeatedly, so its time complexity is **$O(\log n)$** . It works only on sorted data.

For unsorted lists, Linear Search is used.

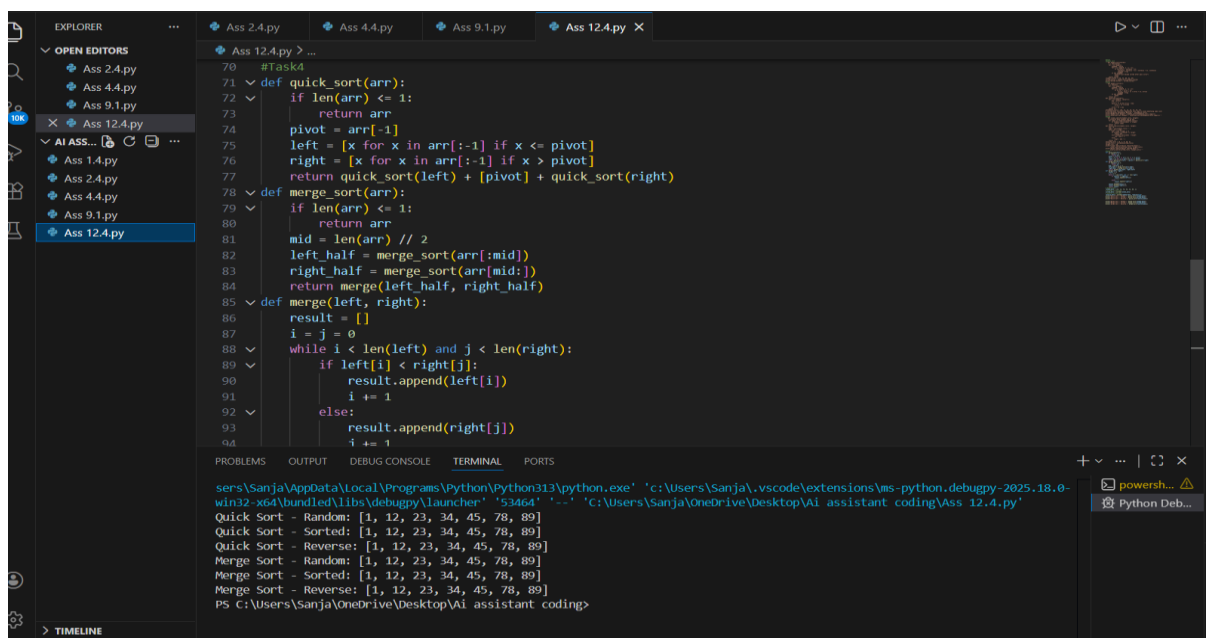
For large sorted lists, Binary Search is faster and more efficient.

Task4:

work with large datasets that may be random, already sorted, or reverse sorted.

1. Complete recursive implementations of Quick Sort and Merge Sort in Python.
2. Add proper docstrings explaining parameters and return values.

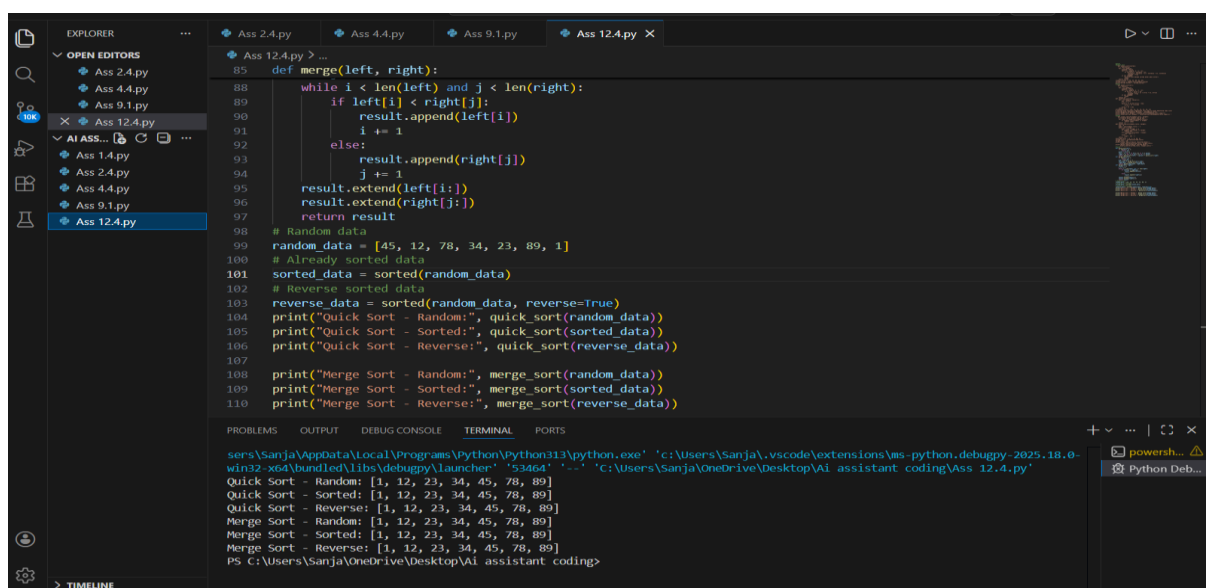
3. Explain how recursion works in both algorithms.
4. Test both algorithms on:
 - Random data
 - Sorted data
 - Reverse sorted data
5. Compare best, average, and worst-case time complexity.
6. Explain practical scenarios where one is preferred over the other.



```
70 #Task4
71 def quick_sort(arr):
72     if len(arr) <= 1:
73         return arr
74     pivot = arr[-1]
75     left = [x for x in arr[:-1] if x <= pivot]
76     right = [x for x in arr[:-1] if x > pivot]
77     return quick_sort(left) + [pivot] + quick_sort(right)
78 def merge_sort(arr):
79     if len(arr) <= 1:
80         return arr
81     mid = len(arr) // 2
82     left_half = merge_sort(arr[:mid])
83     right_half = merge_sort(arr[mid:])
84     return merge(left_half, right_half)
85 def merge(left, right):
86     result = []
87     i = j = 0
88     while i < len(left) and j < len(right):
89         if left[i] < right[j]:
90             result.append(left[i])
91             i += 1
92         else:
93             result.append(right[j])
94             j += 1
```

Terminal Output:

```
PS C:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53464' '--' 'C:\Users\Sanja\OneDrive\Desktop\AI assistant coding\Ass 12.4.py'
Quick Sort - Random: [1, 12, 23, 34, 45, 78, 89]
Quick Sort - Sorted: [1, 12, 23, 34, 45, 78, 89]
Quick Sort - Reverse: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Random: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Sorted: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Reverse: [1, 12, 23, 34, 45, 78, 89]
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding>
```



```
85 def merge(left, right):
88     while i < len(left) and j < len(right):
89         if left[i] < right[j]:
90             result.append(left[i])
91             i += 1
92         else:
93             result.append(right[j])
94             j += 1
95     result.extend(left[i:])
96     result.extend(right[j:])
97     return result
98 # Random data
99 random_data = [45, 12, 78, 34, 23, 89, 1]
100 # Already sorted data
101 sorted_data = sorted(random_data)
102 # Reverse sorted data
103 reverse_data = sorted(random_data, reverse=True)
104 print("Quick sort - Random:", quick_sort(random_data))
105 print("Quick sort - Sorted:", quick_sort(sorted_data))
106 print("Quick sort - Reverse:", quick_sort(reverse_data))
107
108 print("Merge Sort - Random:", merge_sort(random_data))
109 print("Merge Sort - Sorted:", merge_sort(sorted_data))
110 print("Merge Sort - Reverse:", merge_sort(reverse_data))
```

Terminal Output:

```
PS C:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53464' '--' 'C:\Users\Sanja\OneDrive\Desktop\AI assistant coding\Ass 12.4.py'
Quick Sort - Random: [1, 12, 23, 34, 45, 78, 89]
Quick Sort - Sorted: [1, 12, 23, 34, 45, 78, 89]
Quick Sort - Reverse: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Random: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Sorted: [1, 12, 23, 34, 45, 78, 89]
Merge Sort - Reverse: [1, 12, 23, 34, 45, 78, 89]
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding>
```

Observation:

1. Quick Sort has best and average time complexity of $O(n \log n)$.
2. Quick Sort worst case is $O(n^2)$ when pivot selection is poor.
3. Merge Sort has $O(n \log n)$ time complexity in best, average, and worst cases.
4. Quick Sort uses less memory (in-place), while Merge Sort requires extra memory.
5. Quick Sort is usually faster for random datasets.
6. Merge Sort is preferred when guaranteed performance and stable sorting are required.

Task5:

Building a data validation module to detect duplicate user IDs.

1. Write a brute-force duplicate detection algorithm using nested loops.
2. Analyze its time complexity.
3. Suggest an optimized approach using a set or dictionary.
4. Rewrite the algorithm with improved efficiency.
5. Compare performance conceptually for large datasets.
6. Keep explanation simple and clear.

```
111 #Task5
112 def find_duplicates_bruteforce(user_ids):
113     duplicates = []
114     for i in range(len(user_ids)):
115         for j in range(i + 1, len(user_ids)):
116             if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
117                 duplicates.append(user_ids[i])
118
119     return duplicates
120 data = [11, 12, 13, 14, 13, 23]
121 print("Brute-force Duplicates:", find_duplicates_bruteforce(data))
122 #optimized duplicate detection
123 def find_duplicates_optimized(user_ids):
124     seen = set()
125     duplicates = set()
126     # Traverse list only once
127     for uid in user_ids:
128         if uid in seen:
129             duplicates.add(uid)
130         else:
131             seen.add(uid)
132     return list(duplicates)
133 # Example
134 data=[11,12,13,14,13,23,15,80]
135 print("Optimized Duplicates:", find_duplicates_optimized(data))
```

```
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c;; cd 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62614' '--' 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 12.4.py'
Brute-force Duplicates: [13]
Optimized Duplicates: [13]
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
```

Observation:

- 1.The brute-force algorithm uses nested loops to compare every element with all others, resulting in $O(n^2)$ time complexity.
- 2.It performs a large number of unnecessary comparisons, especially for large datasets.
- 3.The optimized algorithm uses a set (or dictionary) to store already seen elements.
- 4.Since set lookup takes $O(1)$ time, the entire list is processed in a single loop, giving $O(n)$ time complexity.
- 5.The performance improved because repeated comparisons were replaced with fast lookups.
- 6.Therefore, the optimized approach is much faster and more suitable for large datasets.